Twinkle — database design and data dictionary

SQLite, one file, sixteen tables.

The columns and types below are taken from the schema the application actually creates — `dotnet ef dbcontext script` — not transcribed from the entity classes. If this document and `src/Pharma.Core/Entities.cs` ever disagree, the code is right and this is stale.

Upgrading a live database is a separate subject: DATABASE_UPGRADES.md.

Read this before the tables

Three things about how SQLite stores this schema will mislead anyone who queries the file directly.

Money is stored as TEXT

`Mrp`, `NetAmount`, `ConsultationFee` and every other `decimal` are **TEXT** columns, not numeric. The EF SQLite provider maps `decimal` that way to keep full precision — SQLite's only numeric types are INTEGER and REAL, and REAL would lose paise.

`OnModelCreating` sets precision 12 and scale 2 on every decimal, and **in SQLite that has no effect at all.** It is honest intent that the provider ignores.

What it means in practice:

```
-- WRONG. Adds text, sorts text: '9.00' comes after '10.00'.
SELECT SUM(NetAmount) FROM Sales ORDER BY NetAmount DESC;

-- Right.
SELECT SUM(CAST(NetAmount AS REAL)) FROM Sales ORDER BY CAST(NetAmount AS REAL) DESC;
```

The application is unaffected — EF converts on the way in and out. Any external reporting tool, Excel import or hand-written SQL **must cast**, or it will silently produce wrong money.

No column has a length

There is no `MaxLength` anywhere in the model, so every string is unbounded TEXT. Nothing stops a 5,000-character patient name or a batch number longer than the printed bill can fit. SQLite would not enforce a length even if one were declared, but EF would, and it does not.

This is a gap rather than a decision. Names, phone numbers, batch numbers and GSTIN all have real-world maximums that are not written down anywhere. It has not caused a problem because the screens are the only way in.

Enums are integers

Every enum is stored as its numeric value, so a raw query returns `1` where the screen says `Cash`. The maps are in Enum values at the end — and note that **the numbers are deliberately fixed**, because changing one would silently reinterpret every existing row.

Where it lives

Database	`C:\HMS\DB\twinkle.db`
Backups	`C:\HMS\DBBackup\` — daily, plus one before every schema change
Logs	`C:\HMS\Logs\`

Outside the program folder on purpose: an upgrade replaces `C:\HMS\App` and must not be able to touch the data. Copying `twinkle.db` to a pen drive is the whole backup. `C:\HMS` rather than `%ProgramData%` because a clinic can find it and someone can read the path out over the phone.

Conventions

Every table ends with the same four columns, from `BaseEntity`:

Column	Type	Null	Description
`Id`	TEXT	no	GUID, primary key, generated by the application
`CreatedAt`	TEXT	no	ISO date-time. Stamped in `SaveChanges`, never by the caller
`UpdatedAt`	TEXT	yes	Stamped on modify. Null means never edited
`IsDeleted`	INTEGER	no	0 or 1. Soft delete

They are omitted from every table below to keep the columns that matter visible.

GUID keys, not integers. Documents carry their own human-readable numbers (`PatientNo`, `BillNo`, `VisitNo`) from the `Counters` table, so nothing needs a readable row id. GUIDs also leave the door open to merging two databases, which an autoincrement forecloses.

Soft delete, filtered explicitly. There is deliberately **no global query filter**; services write `.Where(x => !x.IsDeleted)` themselves. EF warns on required navigations into filtered entities, and in a codebase this size the explicit clause is easier to follow. The cost is that a forgotten `Where` shows deleted rows.

Local time, not UTC. One PC, one timezone, and the day book means "the day the shop had" — UTC would push evening sales onto tomorrow's report. This needs revisiting the day there is a second branch.

OPD

Patients

The register. One row per person, for life.

Column	Type	Null	Description
`PatientNo`	TEXT	no	`P00001` . From `Counters` . **Unique**
`Name`	TEXT	no	Indexed — searched as you type
`Phone`	TEXT	no	Indexed, **not unique**: a family shares one number and a search must return all the children
`Gender`	INTEGER	no	Gender
`Age`	INTEGER	no	Years, entered at the desk. **Not** a date of birth — see the note below
`Address`	TEXT	yes	
`Allergies`	TEXT	yes	Free text. Shown on the consultation screen

> `Age` being a number is wrong for a paediatric clinic over time: a two-year-old's record is stale within a year and nothing recalculates. It is what the desk is given at the counter. Changing it is a migration plus a UI change, so it is recorded here rather than quietly fixed.

Doctors

Column	Type	Null	Description
`Name`	TEXT	no	Appears as an OPD tab and on the prescription
`RegistrationNo`	TEXT	yes	Printed on the prescription. Required on a real one
`Speciality`	TEXT	yes	Printed under the name
`Phone`	TEXT	yes	
`ConsultationFee`	TEXT	no	Decimal-as-text. The **default** that fills in when booking
`IsActive`	INTEGER	no	Hidden from booking when 0

The fee actually charged lives on the visit, so changing a doctor's rate never rewrites history.

Visits

A booking and a visit are the same row. Turning up moves `Status` from `Booked` to `Waiting`; the desk never re-keys anything.

Column	Type	Null	Description
`VisitNo`	TEXT	no	`V00001`. **Unique**
`TokenNo`	INTEGER	no	The number called out. Restarts each day
`ScheduledOn`	TEXT	no	**Indexed** — every queue refresh reads by date
`Status`	INTEGER	no	VisitStatus
`PatientId`	TEXT	no	→ `Patients`, **restrict**
`DoctorId`	TEXT	no	→ `Doctors`, **restrict**
`Complaint`	TEXT	yes	Taken at the desk, before the doctor
`Diagnosis`	TEXT	yes	
`Notes`	TEXT	yes	Advice
`WeightKg`	TEXT	yes	Decimal-as-text. Vitals are all optional
`BloodPressure`	TEXT	yes	Free text — "110/70" is not a number
`TemperatureF`	TEXT	yes	Decimal-as-text
`Fee`	TEXT	no	Charged for this visit, copied from the doctor at booking
`FeePaid`	INTEGER	no	
`FeeReceiptNo`	TEXT	yes	`R00001`. Stored, not derived, so a receipt reprints years later
`FeePaidOn`	TEXT	yes	Needed to reconcile against the day's collection
`FeePaymentMode`	INTEGER	yes	PaymentMode. Null until paid
`FollowUpOn`	TEXT	yes	"Review on"

PrescriptionItems

Column	Type	Null	Description
`VisitId`	TEXT	no	→ `Visits`, **cascade**
`ProductId`	TEXT	**yes**	→ `Products`. Null when the medicine is not stocked here
`MedicineName`	TEXT	no	What is printed, stocked or not
`Dosage`	TEXT	yes	"1-0-1"
`Frequency`	TEXT	yes	

Column	Type	Null	Description
`Days`	INTEGER	no	
`Quantity`	INTEGER	no	Units to dispense
`Instructions`	TEXT	yes	"after food"

> **The nullable `ProductId` is the feature.** A doctor prescribes what the child needs whether or not this pharmacy stocks it; a free-text line prints on the prescription and never becomes a medicine in the catalogue.

Pharmacy

Products — what a medicine *is*

Set up once. **Holds no stock.**

Column	Type	Null	Description
`Name`	TEXT	no	The brand on the pack. **Indexed**
`GenericName`	TEXT	yes	The drug behind the brand. Searched too
`Manufacturer`	TEXT	yes	Part of the duplicate key
`Composition`	TEXT	yes	Salts and strengths, for substituting a brand that has run out
`Storage`	TEXT	yes	"2–8°C, do not freeze". Matters here: vaccines and some syrups are ruined by a warm shelf
`PackSize`	TEXT	yes	As printed — "10 TAB", "100 ML". Free text; shops describe packs their own way
`DispensingUnit`	INTEGER	no	DispensingUnit — what one sellable unit is
`UnitsPerPack`	INTEGER	no	**See the warning below**
`AllowLooseSale`	INTEGER	no	0 for anything that must leave the shop whole
`HsnCode`	TEXT	no	Defaults `3004`. Goes on the tax invoice
`GstRate`	TEXT	no	Decimal-as-text. Percent, defaults 12
`Schedule`	INTEGER	no	DrugSchedule
`RackLocation`	TEXT	yes	Where it sits. Searchable
`ReorderLevel`	INTEGER	no	Below this it appears in Low stock. 0 means never
`IsActive`	INTEGER	no	Hidden from the counter when 0
`SearchKey`	TEXT	no	brand|maker|pack, normalised. **Unique where `IsDeleted = 0`**

> ### UnitsPerPack is the most consequential column in the database
>
> Stock is counted in **sellable units** — tablets, not strips — so part of a strip can be sold. Set this to 1 while `PackSize` says "15 TAB" and the counter charges fifteen times the price for every tablet, and **nothing anywhere reports an error**. That was the fault this system was built to fix.
>
> It is clamped to a minimum of 1 in the property getter, because rows written before that guard existed hold 0. **Settings → Check data health** finds and repairs the disagreement.

> `SearchKey` carries a **unique index filtered on `IsDeleted = 0`**. A duplicate medicine splits its stock across two rows and appears twice at the counter. It is a stored column with a constraint rather than a check in the screen, because a constraint is a guarantee and a screen check is a suggestion. The filter is what lets a deleted medicine's name be used again.

Batches — what is actually on the shelf

Column	Type	Null	Description
`ProductId`	TEXT	no	→ `Products`, **restrict**. Indexed with `BatchNo`
`BatchNo`	TEXT	no	Printed on the pack. Required by law on the bill
`ExpiryDate`	TEXT	no	Good until the **end** of that month
`Mrp`	TEXT	no	Decimal-as-text. **The pack price**, not the unit price
`PurchaseRate`	TEXT	no	Paid per pack
`QtyOnHand`	INTEGER	no	**In base units**, not packs
`UnitsPerPack`	INTEGER	no	Snapshotted from the product at receiving
`SupplierName`	TEXT	yes	Free text — there is no supplier master
`SupplierInvoiceNo`	TEXT	yes	Which bill it came in on. Needed to reconcile
`FreePacks`	INTEGER	no	The "+1" in 10+1
`PacksReceived`	INTEGER	no	Paid-for packs behind this batch
`ReceivedOn`	TEXT	no	
`IsProvisional`	INTEGER	no	1 when keyed at the counter with no supplier bill

Stock is always batch-wise. MRP and expiry are printed on the strip and differ between consignments of the same drug, so both come from the batch and never from the product. A course spanning two batches is two bill lines at two prices.

`UnitsPerPack` is duplicated here **on purpose**: a manufacturer changes pack size between consignments, and old stock must keep pricing against the pack it actually arrived in.

`FreePacks` was captured at receiving and then thrown away, which overstated cost per unit on every margin figure — ten paid for and eleven received makes the real cost `rate × 10 ÷ 11`.

`IsProvisional` is **flagged rather than hidden**: purchases will not tie out against sales until it is matched to a real bill, and that is the *Stock to reconcile* report.

StockEntries — the goods-inward document

Column	Type	Null	Description
`EntryNo`	TEXT	no	**Unique**
`EntryDate`	TEXT	no	
`SupplierName`	TEXT	yes	
`SupplierInvoiceNo`	TEXT	yes	**Indexed** — this is what stops a bill importing twice
`TotalAmount`	TEXT	no	Decimal-as-text
`NetAmount`	TEXT	no	
`DiscountPercent`	TEXT	no	
`Notes`	TEXT	yes	
`ImportedFile`	TEXT	yes	Set when it came from a vendor file rather than being keyed
`ImportProfile`	TEXT	yes	Which profile parsed it
`IsProvisional`	INTEGER	no	Keyed at the counter to get a sale through
`EnteredBy`	TEXT	yes	Windows username. A note, not a control

StockEntryItems

Column	Type	Null	Description
`StockEntryId`	TEXT	no	→ `StockEntries`, **cascade**
`ProductId`	TEXT	no	→ `Products`, **cascade**
`BatchNo`	TEXT	no	
`ExpiryDate`	TEXT	no	
`Quantity`	INTEGER	no	**In packs, as the vendor billed them**
`FreeQuantity`	INTEGER	no	Scheme packs
`UnitsPerPack`	INTEGER	no	Multiplies into base-unit stock
`PurchaseRate`	TEXT	no	Per pack
`Mrp`	TEXT	no	Per pack

This is **the one place packs and units meet**, which is why the receiving screen reads the conversion back out loud: *"20 pack(s) × 15 = 300 tablets onto the shelf"*.

Sales — the bill

Column	Type	Null	Description
`BillNo`	TEXT	no	`INV00001`. **Unique**, and must run without gaps
`BillDate`	TEXT	no	**Indexed** — the day book
`PatientId`	TEXT	yes	→ `Patients`, **set null**. Null for a walk-in
`VisitId`	TEXT	yes	→ `Visits`. Set when raised from an OPD prescription
`CustomerName`	TEXT	no	Free text, defaults `Cash`. **No phone number** — see BILL_REVIEW.md
`DoctorName`	TEXT	yes	Free text, not a `DoctorId` — an outside prescription names a doctor we do not have
`GrossAmount`	TEXT	no	Decimal-as-text
`DiscountAmount`	TEXT	no	Always 0 — the counter gives no discount
`TaxableAmount`	TEXT	no	Back-calculated **out of** the MRP
`CgstAmount`	TEXT	no	
`SgstAmount`	TEXT	no	Not always equal to CGST — see BILL_REVIEW.md
`RoundOff`	TEXT	no	To the nearest rupee
`NetAmount`	TEXT	no	What the customer paid
`PaymentMode`	INTEGER	no	PaymentMode
`Status`	INTEGER	no	SaleStatus. `Returned` is defined but unreachable — there are no returns
`IsTaxInvoice`	INTEGER	no	**Stored, not read from settings**

> `IsTaxInvoice` is on the bill because a reprint years later has to show what was actually given to the customer, even if the clinic has registered for GST since. It is not the same as "carries tax": a registered clinic selling only zero-rated goods still issues a tax invoice.

SaleItems

Column	Type	Null	Description
`SaleId`	TEXT	no	→ `Sales`, **cascade**. The only foreign key on this table
`ProductId`	TEXT	no	**No foreign key.** Plain column
`BatchId`	TEXT	no	**No foreign key.** Plain column

Column	Type	Null	Description
`ProductName`	TEXT	no	Copied at sale time
`BatchNo`	TEXT	no	Copied
`ExpiryDate`	TEXT	no	Copied
`HsnCode`	TEXT	no	Copied
`Quantity`	INTEGER	no	**Base units.** 5 means five tablets, not five strips
`UnitsPerPack`	INTEGER	no	Copied, so a reprint shows the pack as it was
`Mrp`	TEXT	no	Copied from the batch
`DiscountPercent`	TEXT	no	Always 0
`GstRate`	TEXT	no	Copied
`TaxableAmount`	TEXT	no	Computed at sale time and stored
`GstAmount`	TEXT	no	Stored
`LineTotal`	TEXT	no	Stored
`PackLabel`	TEXT	yes	The pack text as printed, e.g. "10 TAB"

> **Bills point at nothing**
>
> `ProductId` and `BatchId` are plain columns with **no foreign key**, and eight fields are copied onto the line.
>
> This is deliberate. **A bill is a document, not a view.** What was handed to the customer must reprint identically in three years, whatever has happened to the medicine since — renamed, repriced, repacked, deleted. A foreign key would also stop a batch ever being tidied away.
>
> The cost is that a medicine renamed today does not change on yesterday's bill. That is the feature, not a defect.

Shared

Counters — document numbering

Column	Type	Null	Description
`Name`	TEXT	no	**Unique.** `bill` , `patient` , `visit` , `receipt`
`Prefix`	TEXT	no	`INV` , `P` , `V` , `R`
`LastNumber`	INTEGER	no	Last issued

Tax invoice numbers must run without gaps, so they come from a row read and incremented in the same transaction as the document — not from a count of existing rows, which would reuse a number after a deletion.

StockAdjustments — the correction trail

Column	Type	Null	Description
`AdjustedOn`	TEXT	no	**Indexed**
`BatchId`	TEXT	no	→ `Batches` , **restrict**
`ProductId`	TEXT	no	→ `Products` , **restrict**
`ProductName`	TEXT	no	Copied, so the trail survives later edits
`BatchNo`	TEXT	no	Copied
`QuantityBefore`	INTEGER	no	
`QuantityAfter`	INTEGER	no	
`Reason`	INTEGER	no	AdjustmentReason
`Notes`	TEXT	yes	
`AdjustedBy`	TEXT	yes	Windows username

Stock otherwise only moves by receiving or selling, and both leave a document. A manual correction has none, **so it writes one** — otherwise a shortfall is indistinguishable from theft and nobody can answer what happened.

H1Register — statutory

Column	Type	Null	Description
`SoldOn`	TEXT	no	
`BillNo`	TEXT	no	Copied, not a foreign key
`ProductName`	TEXT	no	Copied
`BatchNo`	TEXT	no	Copied
`Quantity`	INTEGER	no	
`PatientName`	TEXT	no	
`DoctorName`	TEXT	yes	The prescriber. The whole point of the register

Schedule H1 sales, retained three years by law. A table rather than a query over sales, because it must be producible on demand and must not change when a bill is edited or a medicine reclassified.

Settings — shop identity

Column	Type	Null	Description
`Key`	TEXT	no	**Unique**. `shop.name`, `shop.gstin`, `ui.theme`, …
`Value`	TEXT	no	Everything as text, parsed on read

In the database rather than `appsettings.json`, so a second branch is a data change rather than a redeploy — and why a ClickOnce update replacing `appsettings.json` costs nothing.

ImportProfiles

Column	Type	Null	Description
`Name`	TEXT	no	**Unique**. The supplier
`Description`	TEXT	yes	
`ColumnMap`	TEXT	no	Their column headings mapped to ours
`DateFormats`	TEXT	no	Pipe-separated candidates, tried in order
`ExpiryFormats`	TEXT	no	Same. `MM/yy` and `MM/yyyy` both appear in the wild
`DefaultGstRate`	TEXT	no	Used when their file omits it
`IsActive`	INTEGER	no	

VendorProductCodes

Column	Type	Null	Description
`VendorProfile`	TEXT	no	**Unique with `Code`**
`Code`	TEXT	no	The vendor's own code for the medicine
`ProductId`	TEXT	no	→ `Products`, **cascade**

Two suppliers call the same drug `000071` and `31435`, so the mapping is per vendor. Recording it on the first import is what makes every later import of that vendor match unasked.

Foreign keys and delete behaviour

From	To	On delete	Why
`Visits.PatientId`	Patients	RESTRICT	A patient with visits cannot be removed; the history is the point

From	To	On delete	Why
`Visits.DoctorId`	Doctors	RESTRICT	Past visits name the doctor who saw them
`PrescriptionItems.VisitId`	Visits	CASCADE	A prescription has no meaning without its visit
`PrescriptionItems.ProductId`	Products	NO ACTION	Nullable; a free-text line has none
`Batches.ProductId`	Products	RESTRICT	Never lose stock by tidying the catalogue
`StockEntryItems.StockEntryId`	StockEntries	CASCADE	Lines belong to the document
`StockEntryItems.ProductId`	Products	CASCADE	
`Sales.PatientId`	Patients	**SET NULL**	Removing a patient must not delete their bills
`Sales.VisitId`	Visits	NO ACTION	
`SaleItems.SaleId`	Sales	CASCADE	
`StockAdjustments.BatchId`	Batches	RESTRICT	The audit trail outlives tidying up
`StockAdjustments.ProductId`	Products	RESTRICT	
`VendorProductCodes.ProductId`	Products	CASCADE	A mapping to a deleted product is noise

`SaleItems.ProductId` and `SaleItems.BatchId` appear in no row above — they are not foreign keys at all.

Indexes

Table	Index	Unique	For
Patients	`PatientNo`	yes	The number on the card
Patients	`Name`	no	Counter and desk search
Patients	`Phone`	no	The other way the desk searches
Visits	`VisitNo`	yes	
Visits	`ScheduledOn`	no	The day's queue, every refresh
Visits	`PatientId` , `DoctorId`	no	FK indexes
PrescriptionItems	`VisitId` , `ProductId`	no	FK indexes
Products	`Name`	no	Search as you type
Products	`SearchKey` **where** `IsDeleted = 0`	yes	No duplicate medicines
Batches	`(ProductId, BatchNo)`	no	FEFO allocation and receiving
Sales	`BillNo`	yes	Reprint

Table	Index	Unique	For
Sales	`BillDate`	no	Day book
Sales	`PatientId` , `VisitId`	no	FK indexes
SaleItems	`SaleId`	no	
StockEntries	`EntryNo`	yes	
StockEntries	`SupplierInvoiceNo`	no	Stops a bill importing twice
StockEntryItems	`StockEntryId` , `ProductId`	no	
StockAdjustments	`AdjustedOn`	no	The corrections list
StockAdjustments	`BatchId` , `ProductId`	no	
VendorProductCodes	`(VendorProfile, Code)`	yes	Per-vendor mapping
Settings	`Key`	yes	
Counters	`Name`	yes	
ImportProfiles	`Name`	yes	

Enum values

Stored as integers. **The numbers are fixed** — changing one silently reinterprets every existing row.

Gender

`1` Male · `2` Female · `3` Other

VisitStatus

`1` Booked · `2` Waiting · `3` InConsultation · `4` Completed · `5` Cancelled

PaymentMode

`1` Cash · `2` Upi · `3` Card

There is no Credit and no part payment: every sale is settled in full at the counter, so nothing in the system carries an outstanding balance. The numbers were pinned when a Credit option was removed, so existing rows kept their meaning.

DrugSchedule

`0` None · `1` H · `2` H1 · `3` X

A real chemist's bill shows `G & H` against one line — this cannot express a combination, and has no Schedule G or S. See BILL_REVIEW.md.

DispensingUnit

`1` Tablet · `2` Capsule · `3` Bottle · `4` Sachet · `5` Tube · `6` Vial · `7` Piece

AdjustmentReason

`1` Recount · `2` Breakage · `3` Expired · `4` Lost · `5` EntryError · `6` Other

SaleStatus

`1` Completed · `2` Returned · `3` Cancelled

`Returned` is defined but unreachable — there are no sales returns in this version.

Things that look like columns and are not

Query the file directly and these are missing. They are computed in the application and explicitly `Ignore` d in `OnModelCreating` :

Entity	Computed
Product	`StockOnHand` , `PackDescription` , `UnitPriceLabel` , `PackPriceLabel` , `RackLabel` , `Shortage` , `Level`
Batch	`IsExpired` , `UnitPrice` , `OnHand` , `Returnable` , `DaysToExpiry` , `EffectivePackCost` , `Display`
Visit	`IsWaiting` , `PatientLine` , `FeeBadge` , `RowSummary` , `WaitedFor`
Patient	`Display`
SaleItem	`QuantityDescription`
StockEntryItem	`LineTotal` , `UnitsReceived`
StockAdjustment	`Change` , `Direction`
ImportProfile	`SplitDateFormats` , `SplitExpiryFormats`

`StockOnHand` is the one to know. There is no stock column on `Products` at all — stock is the sum of a product's batches, always, so it cannot drift out of step with them:

```
SELECT p.Name, SUM(b.QtyOnHand) AS OnHand
FROM Products p
LEFT JOIN Batches b ON b.ProductId = p.Id AND b.IsDeleted = 0
WHERE p.IsDeleted = 0
GROUP BY p.Id;
```

What this schema does not do

Recorded so nobody has to discover it.

Not supported	Note
Sales returns	No credit note, no reverse entry. `SaleStatus.Returned` is unreachable. Do it on paper and correct the count with a reason
A stock ledger	`QtyOnHand` is a running figure. Adjustments are recorded, but there is no movement table, so the shelf cannot be reconstructed as of an arbitrary past date
IGST / inter-state	CGST and SGST only
A supplier master	Supplier is free text on the batch and the entry, so the same supplier can be spelled three ways
Users, logins, permissions	Everyone on the PC has the same rights. `EnteredBy` and `AdjustedBy` hold the Windows username — a note, not a control
More than one PC	One file, one till. No concurrency design beyond SQLite's own locking
Column lengths	No `MaxLength` anywhere
Date of birth	`Age` is a number, entered once
Multiple schedules per medicine	One value only
Customer phone on a bill	Only a free-text name

Changing it

Never by hand, never with a SQL script run at a clinic:

```
dotnet ef migrations add DescribeTheChange --project src\Pharma.Data --startup-project src\Pharma.Data
```

The rules, the tests behind the upgrade path, and what SQLite makes awkward are in DATABASE_UPGRADES.md.

To regenerate the authoritative DDL this document was written from:

```
dotnet ef dbcontext script --project src\Pharma.Data --startup-project src\Pharma.Data --output schema.sql
```